

Recap tecnico delle modifiche implementate nella pipeline topologica

1. Obiettivo generale del lavoro

L'obiettivo delle modifiche introdotte in questa fase è stato rendere la pipeline capace non solo di stimare la posizione geometrica dei terminali, ma anche di attribuire ai terminali stessi una semantica elettrica più leggibile, di costruire reti topologiche più coerenti e di produrre output finali più utili sia per l'analisi umana sia per l'utilizzo da parte di un LLM.

In particolare, il lavoro si è concentrato su quattro linee principali:

1. miglioramento della stima dei terminali nei componenti più complessi o ambigui;
2. introduzione di una semantica esplicita dei terminali per componenti polarizzati o direzionali;
3. robustezza della costruzione delle net e del matching terminale→net, soprattutto nei casi di supply implicite e pin ausiliari degli operational amplifier;
4. esportazione finale del risultato in forme più leggibili: graph JSON, semantic explanation JSON e LLM context markdown.

Nel complesso, la pipeline evolve da una rappresentazione principalmente geometrica a una rappresentazione topologico-semantica del diagramma.

2. Visione d'insieme della pipeline aggiornata

La pipeline, nella configurazione attuale, può essere riassunta così:

- 01_detect_components: detection YOLO dei componenti
- 02_assign_instances: assegnazione degli identificativi di istanza
- 03_estimate_terminals: stima dei terminali, dell'orientazione e, dove possibile, della semantica dei terminali
- 04_extract_wires: estrazione dei fili e skeletonizzazione
- 05_build_nets: costruzione delle candidate net a partire dallo skeleton e dai terminali
- 06_match_terminals_to_nets: assegnazione finale di ogni terminale a una net
- 07_export_graph: esportazione del risultato come grafo e come rappresentazione semantica descrittiva

- 08_visualize_graph: visualizzazioni statiche/interattive e dashboard finale

La parte più rilevante del lavoro odierno ha riguardato soprattutto i passi 03, 05, 06, 07 e 08, insieme all'aggiornamento dei metadati nel file `class_terminals_v1.yaml` e all'introduzione del modulo `export_semantic_explanation.py`.

3. Aggiornamento dei metadati delle classi (`class_terminals_v1.yaml`)

3.1 Ruolo del file YAML

Il file `class_terminals_v1.yaml` è diventato il punto centrale per descrivere, per ogni classe:

- il tipo simbolico (`one_terminal`, `two_terminal`, `three_terminal`, `multi_terminal`, `variable_terminal`)
- se la classe deve essere usata per il masking e/o per la stima dei terminali
- la strategia di stima dei terminali (`terminal_strategy`)
- l'orientazione di default e le configurazioni dei terminali per ogni orientazione
- l'eventuale strategia di semantizzazione dei terminali (`semantic_terminal_strategy`)
- i ruoli semantici previsti (`semantic_roles`)

Questo cambiamento è importante perché sposta la conoscenza strutturale dei componenti fuori dal codice hard-coded e la rende configurabile nei metadati.

3.2 Terminali nominali e terminali semantici

Nel sistema attuale esistono due livelli di naming:

- terminale nominale/geometrico: nomi come `t1`, `t2`, `t3`, `in1`, `in2`, `out`, `aux1`, `aux2`
- terminale semantico: nomi come `positive`, `negative`, `anode`, `cathode`, `G`, `S`, `D`, `B`, `C`, `E`

Il primo livello serve per la localizzazione geometrica coerente con l'orientazione. Il secondo serve per produrre una rappresentazione elettrica più leggibile e più utile per il ragionamento topologico. Questo secondo livello viene poi copiato anche nei campi `display_name` e `display_terminal_id`, in modo che gli output finali mostrino direttamente il nome semanticamente corretto quando disponibile.

3.3 Classi a due terminali polarizzati o direzionali

Sono stati introdotti `semantic_terminal_strategy` e `semantic_roles` per diversi componenti a due terminali:

- Battery: **battery_positive_from_long_plate**, con ruoli positive e negative
- Current_Source: **current_source_direction_from_arrow**, con ruoli current_to e current_from
- Diode: **diode_cathode_from_bar**, con ruoli cathode e anode
- LED: stessa semantica del diodo, quindi cathode e anode
- Polarized_Capacitor: **polarized_capacitor_positive_from_marker**, con ruoli positive e negative
- Voltage_Source: **voltage_source_positive_from_plus_marker**, con ruoli positive e negative

Questa estensione permette alla pipeline di distinguere tra terminali geometricamente equivalenti ma elettricamente non equivalenti.

3.4 Componenti a tre terminali con semantica esplicita

Per i componenti a tre terminali sono stati aggiunti ruoli semantici espliciti:

- Mosfet: semantic_terminal_strategy = mosfet_gate_with_optional_source_drain, con ruoli G, S, D
- NPN_Transistor: semantic_terminal_strategy = npn_emitter_from_arrow_branch, con ruoli B, E, C

3.5 Operational amplifier

Per l'Operational_Amplifier è stata definita una struttura multi-terminale con:

- in1
- in2
- out
- aux1 opzionale
- aux2 opzionale

Questa definizione è esplicitata per tutte le orientazioni (right, left, top, bottom) e i pin ausiliari sono marcati come optional. In questo modo l'operazionale può essere trattato come un componente con tre terminali obbligatori e fino a due terminali di supply opzionali.

3.6 Classe Terminal

La classe Terminal è stata impostata come variable_terminal con strategia terminal_auto_one_or_two. Ciò significa che il simbolo può produrre automaticamente uno o due terminali a seconda dei lati realmente connessi.

4. Modifiche al passo 03: stima dei terminali

4.1 Obiettivo del passo 03

Il passo 03 stima i terminali di ogni componente rilevato nel passo 02. Nella versione aggiornata, non si limita a produrre coordinate (x, y), ma restituisce anche:

- i terminali con `terminal_id` univoco
- l'orientazione stimata del componente
- eventuali score diagnostici
- la modalità con cui il punto terminale è stato calcolato
- il naming semantico del terminale, quando disponibile

Inoltre il dataset della pipeline è ora parametrizzato tramite la variabile d'ambiente `PIPELINE_DATASET`, evitando di dover modificare manualmente i path a ogni esperimento.

4.2 Dispatcher delle strategie

Il file [dispatcher.py](#) centralizza la scelta della strategia in base a `terminal_strategy` e alla classe del componente. In particolare:

- per `auto_by_aspect_ratio`, l'orientazione viene dedotta dal bbox, con gestione specifica di componenti come induttori e trasformatori
- per `one_terminal_by_orientation`, il lato attivo viene rilevato dai pixel di foreground
- per i componenti a due terminali vengono selezionate strategie specializzate per capacitori, switch, LED/diodi, sorgenti circolari e resistori variabili
- per gli opamp viene usata una strategia dedicata `opamp_by_orientation_and_optional_supply`
- per i tre terminali viene usata `three_terminal_by_side_pattern`

4.3 Point modes e localizzazione dei terminali

Nel file [processor.py](#), la posizione finale del terminale viene calcolata con diverse modalità (`terminal_point_mode`):

- `bbox_side_center` per i casi semplici
- `two_terminal_side_peak` quando serve cercare il picco di connessione lungo un lato
- `three_terminal_structured` per componenti a tre terminali
- modalità dedicata per opamp

Per ciascun terminale viene salvato anche `terminal_point_debug`, utile per capire in seguito come il punto è stato stimato.

4.4 Naming semantico nei terminali esportati

Nel processor ogni terminale viene inizialmente creato con:

- `terminal_id` geometrico
- `name` nominale
- `display_name` inizialmente uguale al nome nominale
- `display_terminal_id` inizialmente uguale a `instance_id:name`

Successivamente, le funzioni di semantic resolution possono aggiornare `display_name` e `display_terminal_id` con il nome semantico (G, D, S, positive, negative, anode, cathode, ecc.), in modo che gli output successivi mostrino direttamente l'informazione più utile.

4.5 Semantica dei componenti a due terminali

Il file `semantic_two_terminal.py` introduce una fase di semantizzazione per i componenti a due terminali. La logica generale è:

1. si stimano score per ciascun lato del simbolo
2. si decide quale lato è il lato marker e quale l'altro
3. si assegnano i ruoli previsti dallo YAML
4. si popolano campi come `semantic_terminal_name`, `semantic_terminal_id`, `semantic_slot`, `semantic_confidence`, `semantic_resolution_mode`, `semantic_evidence_type`, `semantic_role_family`, `semantic_polarity` o `semantic_direction`

Le principali strategie implementate sono:

- lato della barra per diodi e LED
- lato marcato per polarized capacitor
- piastra lunga per battery
- simbolo + per voltage source
- distribuzione interna coerente con la freccia per current source

4.6 Semantica dei componenti a tre terminali

Nel file `strategies_three_terminal.py` la stima non si limita a scegliere il lato singolo del componente, ma usa anche probe strutturati per distinguere semanticamente i rami.

Per i MOSFET sono stati introdotti:

- score dedicati per riconoscere il lato gate
- probe specifici per il ramo con freccia
- combinazione di probe interni ed esterni
- assegnazione semantica G, S, D solo se la confidenza della distinzione source/drain è sufficiente

Per gli NPN è stata introdotta la logica `npn_emitter_from_arrow_branch`, con:

- riconoscimento del lato singolo come base
- probe del ramo con freccia per identificare l'emettitore
- uso di un fallback dedicato quando la confidenza del probe standard non è sufficiente

4.7 Riduzione dell'influenza del testo

Una parte fondamentale del lavoro ha riguardato la riduzione dell'effetto del testo adiacente al simbolo. Sono stati introdotti binary locali ripuliti basati su connected components:

- per la classe Terminal, così da evitare che scritte vicine influenzino l'identificazione di uno o due lati reali
- per i componenti a tre terminali, così da evitare che testo come M5, Q1, valori, sigle o label si attacchino alla struttura del simbolo e spostino i terminali stimati

4.8 Gestione specifica degli opamp

Nel passo 03, dopo la stima dei terminali di tutti i componenti, viene eseguito `snap_opamp_top_aux_to_nearby_terminal(...)`, una correzione dedicata agli auxiliary supply pin degli operational amplifier.

5. Modifiche alle probe functions ([probes.py](#))

Il file [probes.py](#) raccoglie molte delle primitive che permettono di stimare orientazioni, lati attivi e caratteristiche locali dei simboli.

Tra i punti principali:

- probe locali per i lati del bbox
- probe multi-anchor per casi come switch o strutture più complesse
- probe stretti per LED
- probe near/far per il lato singolo dei MOSFET

- score combinati per capire se il gate del MOSFET sia a sinistra o a destra nei casi speculari

6. Modifiche alla classe Terminal

La classe Terminal richiede una gestione particolare perché il simbolo può rappresentare:

- un solo terminale
- due terminali contrapposti
- un terminale con testo vicino
- un terminale con wire molto corto
- un terminale con frammenti esterni o discontinuità

Nel file `strategies_terminal_class.py` è stata implementata una logica più prudente:

1. default semantico = un terminale
2. si accetta l'ipotesi di due terminali solo se l'evidenza è forte
3. si applica una pulizia locale del binary per ridurre il rumore testuale
4. si combinano score locali e far scores
5. si usa anche una prior geometrica leggera, ma solo quando il bbox non è quasi quadrato

7. Modifiche al passo 05: costruzione delle net

7.1 Obiettivo del passo 05

Il passo 05 costruisce le net candidate a partire dallo skeleton estratto nel passo 04. Il flusso è:

1. connected components dello skeleton
2. matching locale terminale → label candidata
3. costruzione delle candidate net
4. filtraggio delle net candidate
5. relabeling delle net mantenute
6. salvataggio di label map, overlay e immagini debug

7.2 Matching standard dei terminali sullo skeleton

Per il matching dei terminali allo skeleton, il passo 05 usa:

- una finestra direzionale coerente con `relative_position` del terminale
- uno snap al pixel etichettato più vicino

- un fallback quadrato quando la ricerca direzionale non trova nulla

7.3 Gestione dedicata degli auxiliary supply degli opamp

È stata introdotta una strategia specifica che:

- estende la finestra di ricerca lungo il corridoio verticale del pin
- assegna un punteggio ai label candidati in base a reach, area, gap orizzontale e penalità sul lato sbagliato
- privilegia label già ancorate a terminali non-ausiliari quando esistono

7.4 Supply implicite e merge di frammenti orfani

Uno degli aggiornamenti più importanti del passo 05 è la gestione delle implicit supply. In molti diagrammi il pin di alimentazione dell'opamp non è collegato a un vero simbolo di terminale esterno, ma solo a uno stelo o a un frammento di linea.

Per questi casi è stata introdotta una logica che:

- cerca un frammento vicino al pin ausiliario
- verifica che esista un'estensione coerente nella direzione outward
- concatena più source labels se formano una piccola catena verticale coerente
- marca il match come `is_implicit_supply = True`

Da qui derivano due concetti chiave:

- `merged_source_labels`: una net finale può essere composta dall'unione di più source labels originarie
- `implicit_aux_merges`: il passo 05 salva esplicitamente quali merge impliciti sono stati eseguiti

7.5 Output del passo 05

Il passo 05 salva, oltre alle net, una notevole quantità di debug:

- `label_map_path`
- `terminal_to_candidate_labels`
- `rejected_candidates`
- overlay delle net
- terminal debug image
- statistiche sul numero di label, terminali e net mantenute

8. Modifiche al passo 06: matching terminali → net finali

8.1 Obiettivo del passo 06

Il passo 06 prende il risultato del passo 05 e assegna a ogni terminale una net finale.

8.2 Mappa source label → net index

Una modifica importante è che la mappa `source_label_to_net_index` non usa solo la `source_label` principale, ma anche `merged_source_labels`.

8.3 Search plan multi-stage

Per ogni terminale viene provata una sequenza di stadi di ricerca:

- `directional_primary`
- `circle_primary`
- `directional_fallback`
- `circle_fallback`

8.4 Gestione dedicata degli auxiliary pin degli opamp

Per gli auxiliary pin degli opamp è stato introdotto uno stadio dedicato `run_opamp_aux_vertical_stage(...)`, che cerca label in una capsula verticale attorno al terminale.

Inoltre, se un auxiliary pin è stato nel passo 03 snappato a un terminale vicino, il passo 05/06 può ereditare la label di quel terminale adiacente.

8.5 Stati del match e classificazione della fiducia

Il passo 06 distingue chiaramente tra:

- `matched_preferred`
- `matched_implicit_supply`
- `unmatched`

La funzione di classificazione della confidenza genera anche warning come:

- `distance_too_large`

- no_preferred_net_from_05
- matched_without_preferred_label
- used_fallback_search
- used_circle_search
- used_opamp_aux_vertical_search

8.6 Informazioni salvate per ogni terminale

Dopo il passo 06, ogni terminale può contenere:

- net matched finale
- preferred net derivata dal passo 05
- informazione se la net è implicita
- search stage e search window usati
- distanza di snap
- stato del match
- confidenza
- warnings

9. Modifiche al passo 07: export del grafo e rappresentazioni semantiche

9.1 Obiettivo del passo 07

Il passo 07 trasforma il risultato topologico del passo 06 in una rappresentazione strutturata riutilizzabile. Gli output previsti sono:

- graph JSON
- semantic explanation JSON
- LLM context markdown
- nodes CSV
- edges CSV
- CSV combinati di batch

9.2 Graph JSON come fonte tecnica principale

Il graph JSON è la rappresentazione tecnica di riferimento. Il modello dati è:

- Diagram -> HAS_COMPONENT -> Component

- Diagram -> HAS_NET -> Net
- Component -> HAS_TERMINAL -> Terminal
- Terminal -> CONNECTED_TO -> Net

9.3 Dal simplified JSON al semantic_explanation.json

Nel corso del lavoro era stato introdotto un simplified JSON per avere una forma più leggibile del diagramma. Nella versione attuale questa idea è stata consolidata e generalizzata: il formato attivo non è più il vecchio simplified JSON, ma semantic_explanation.json.

Il codice mantiene una funzione build_simplified_diagram_json(...) solo come wrapper di compatibilità legacy, ma l'export reale usa build_semantic_explanation(...).

9.4 Contenuto di semantic_explanation.json

Il file semantic_explanation.json contiene una descrizione deterministica del diagramma organizzata in sezioni. Le principali sono:

- diagram_metadata
- summary
- component_descriptions
- net_descriptions
- branch_summaries
- component_to_component_relations
- component_relation_groups
- functional_paths
- structural_patterns
- terminal_facts

9.5 Ruoli descrittivi di componenti e net

Il modulo export_semantic_explanation.py costruisce inferenze descrittive caute sui ruoli di componenti e net.

9.6 Functional paths e pattern strutturali

Tra le sezioni più utili per un LLM ci sono anche:

- functional_paths
- structural_patterns

10. Creazione del markdown human-comprehensible (llm_context.md)

Il file llm_context.md è prodotto da build_semantic_llm_context(...) e rappresenta una versione testuale leggibile da umano e da LLM.

Le sezioni principali del markdown sono:

- Purpose
- Overview
- Main Branches
- Component Descriptions
- Net Descriptions
- Important Component-to-Component Relations
- Functional Paths
- Structural Patterns
- Terminal Facts
- Companion Files

La logica di questo markdown è volutamente descrittiva e prudente: non inventa guasti e non fa diagnosi automatiche aggressive.

11. Modifiche al passo 08: visualizzazione del grafo

Il passo 08 genera le visualizzazioni finali della topologia esportata. Le viste prodotte sono:

- full graph
- component → net graph
- overlay sul diagramma
- dashboard HTML batch

In più, la versione aggiornata copia nella cartella downloads non solo i graph JSON, ma anche semantic_explanation.json e llm_context.md.

12. Output finali prodotti dalla pipeline aggiornata

Alla fine del processo, per ogni diagramma sono ora disponibili tre livelli di output:

1. livello tecnico: graph JSON

2. livello semantico strutturato: semantic_explanation.json

3. livello testuale human/LLM readable: llm_context.md

13. Placeholder per le immagini della nuova esecuzione

Figura 1 — Terminali stimati nel passo 03

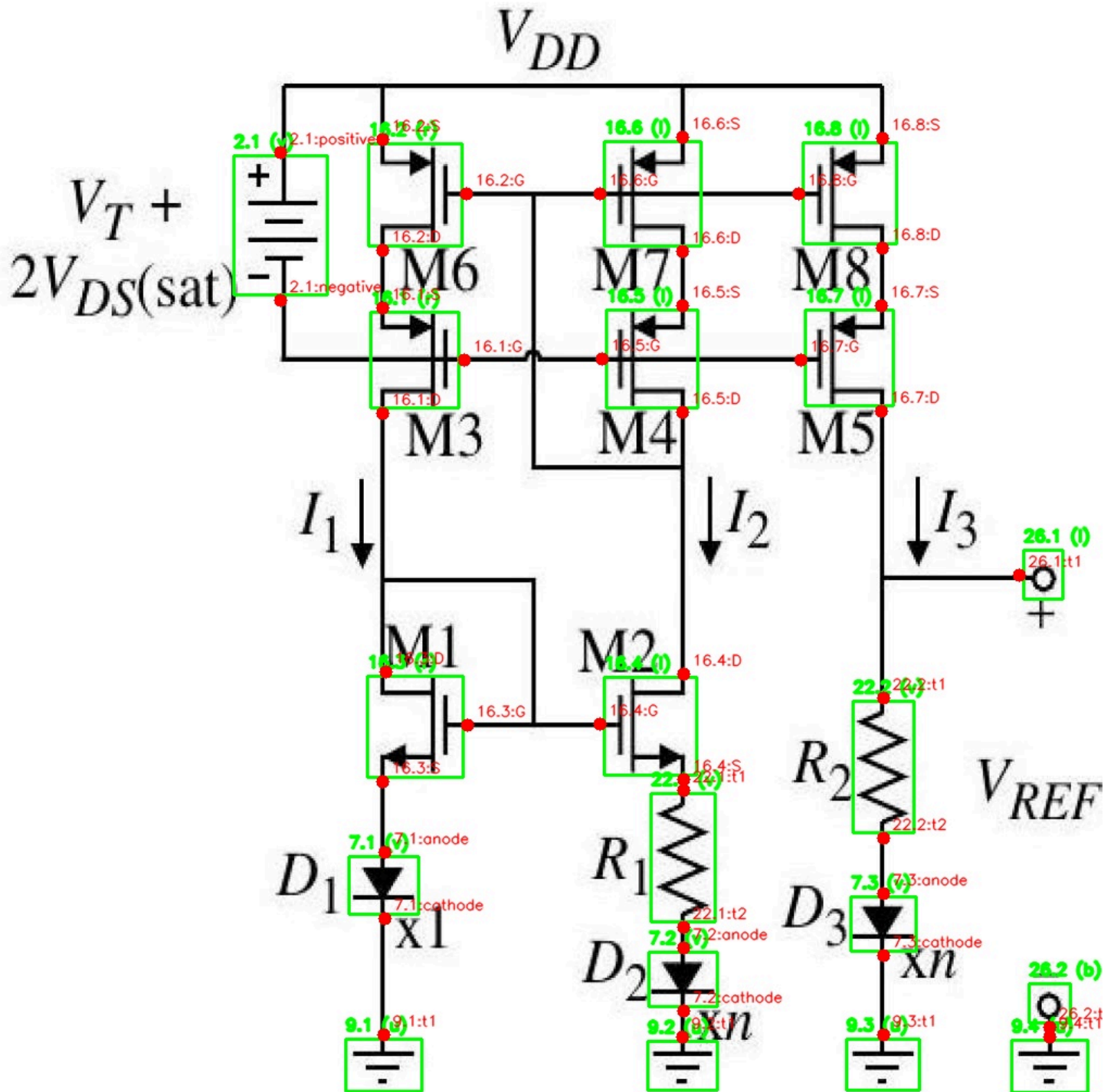


Figura 2 — Costruzione delle net nel passo 05

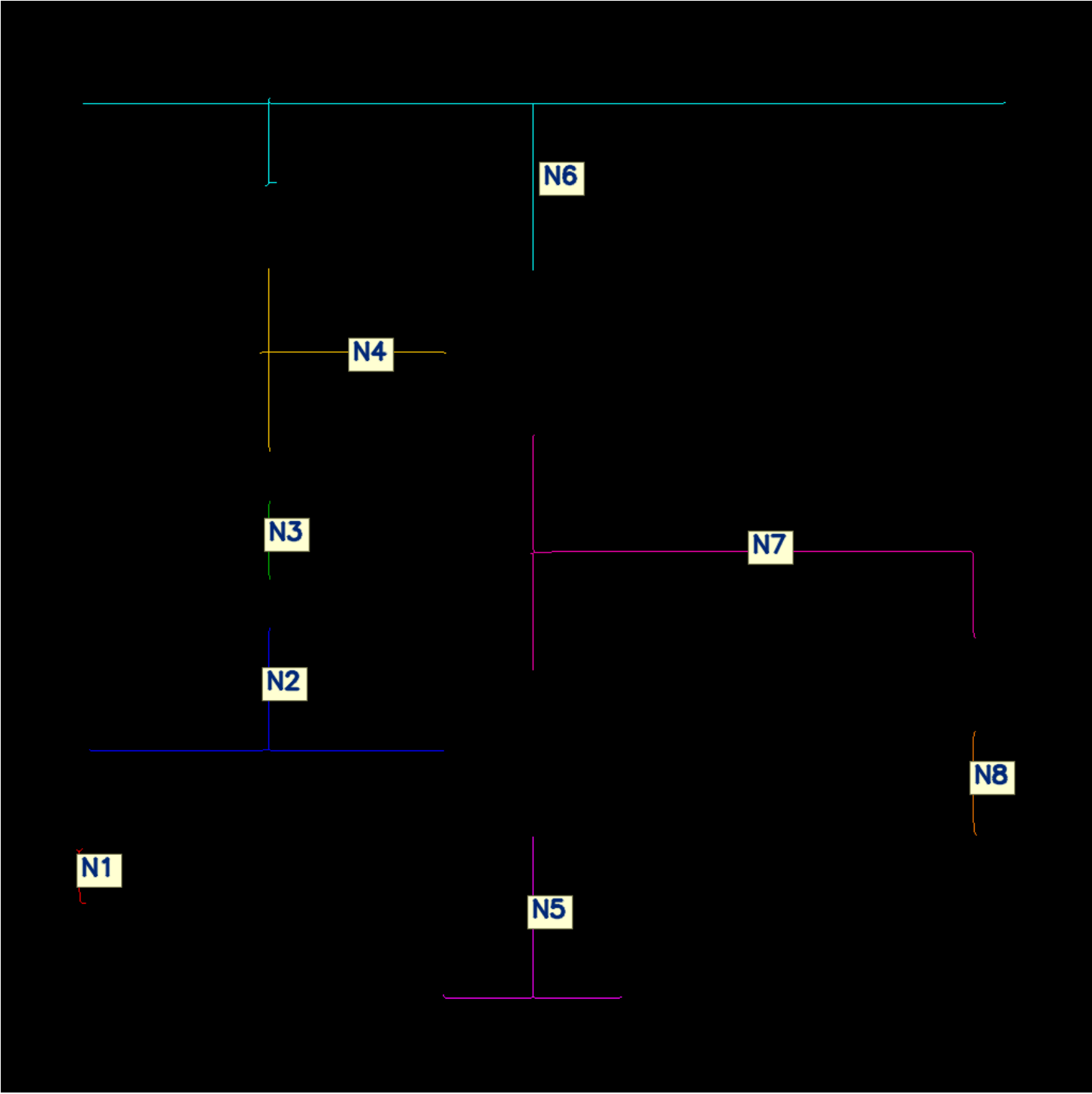


Figura 3 — Matching terminale → net nel passo 06

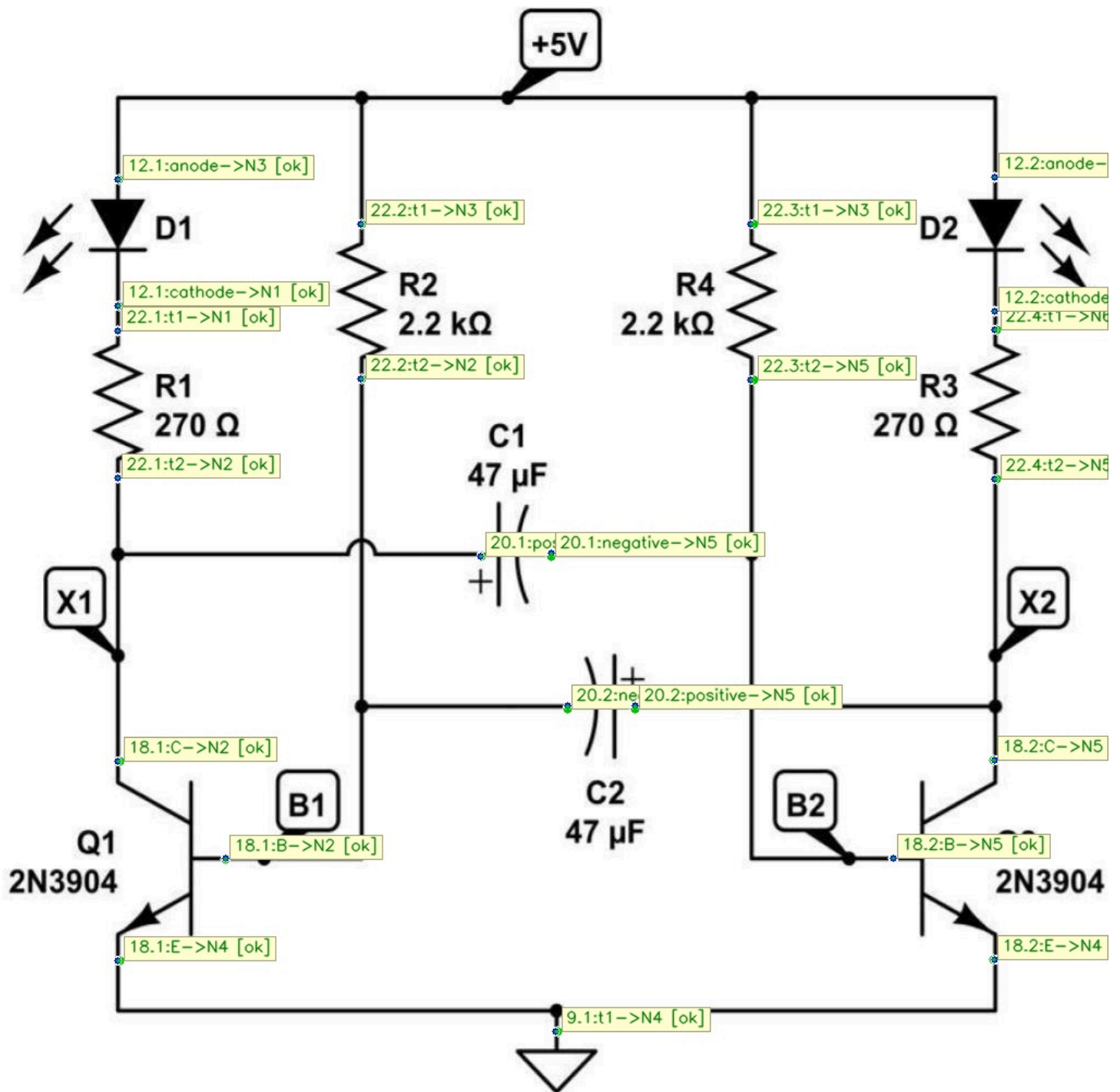


Figura 4 — Overlay topologico finale

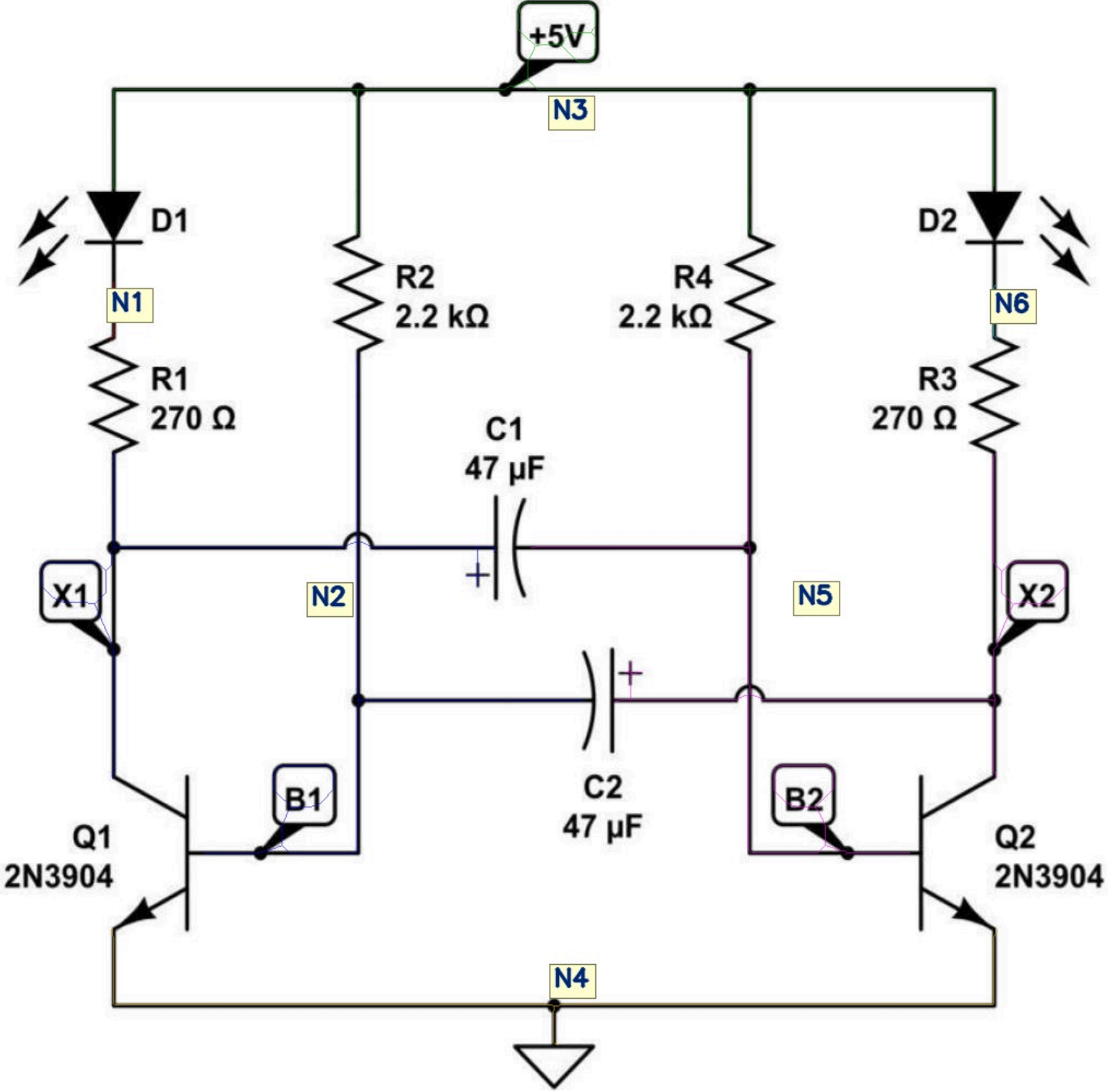
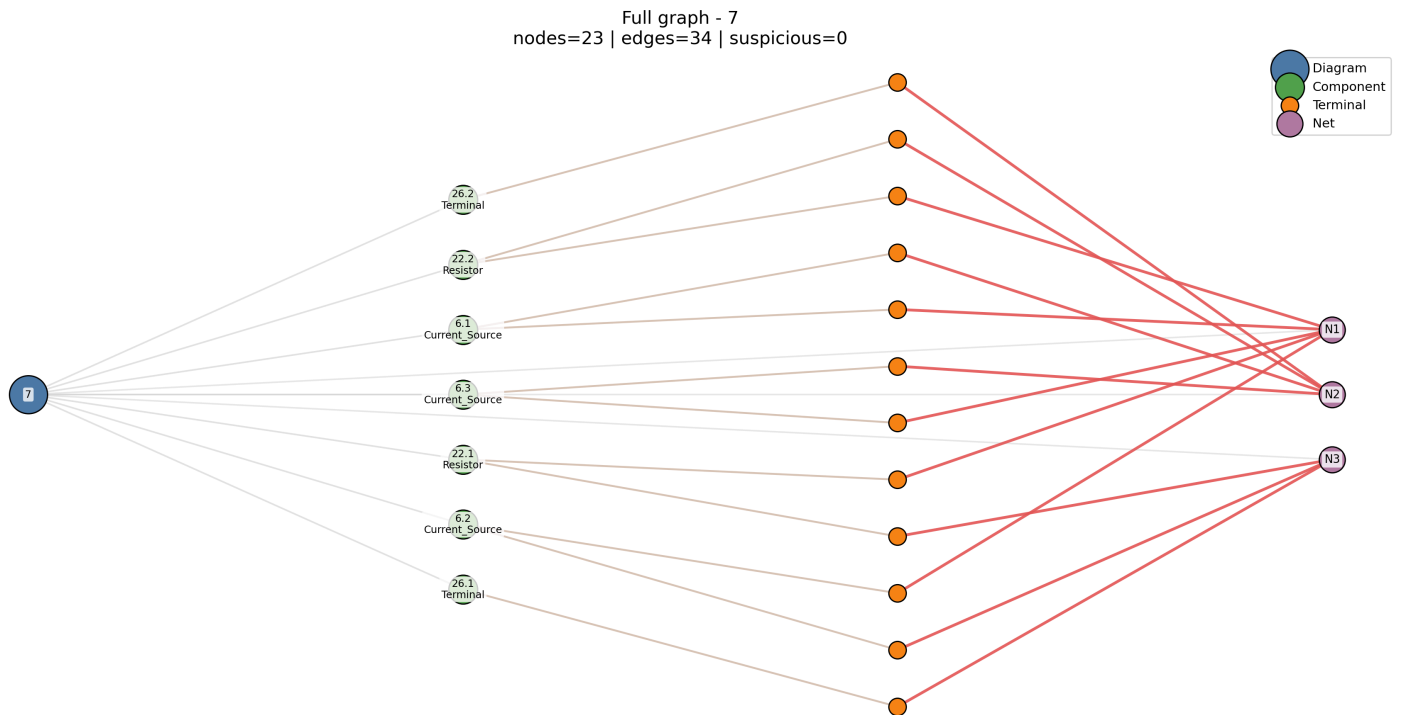


Figura 5 — Grafo finale del diagramma



14. Sintesi finale utile per la tesi

Le modifiche introdotte in questa fase hanno trasformato la pipeline da semplice estrattore di componenti e connessioni in un sistema capace di produrre una rappresentazione topologica più robusta, più interpretabile e più vicina al significato elettrico dei simboli.

Dal punto di vista metodologico, i contributi principali possono essere riassunti così:

- definizione configurabile dei terminali e della loro semantica nel file YAML
- localizzazione più robusta dei terminali tramite probe dedicati e text suppression locale
- supporto avanzato a operational amplifier, MOSFET, transistor NPN e componenti polarizzati
- gestione delle supply implicite e dei merge di frammenti di net
- esportazione finale del risultato in tre forme complementari: tecnica (graph JSON), strutturata semantica (semantic_explanation.json) e testuale (llm_context.md)

Questa evoluzione è particolarmente rilevante in prospettiva dell'uso di un LLM, perché consente di fornire al modello non soltanto l'immagine del diagramma, ma anche una rappresentazione strutturata e descrittiva del circuito, con un livello di ambiguità inferiore rispetto al puro input visivo.